FACULTAD DE INGENIERÍA Y ARQUITECTURA

CARRERA PROFESIONAL DE INGENIERÍA DE SOFTWARE

EXAMEN PARCIAL

APELLIDOS Y NOMBRES	ROGER INFA SANCHEZ
CURSO	Web Semánticas y Ontologías
SEMESTRE	Octavo
DOCENTE	Josue Miguel Flores Parra
FECHA	22-10-2026

Todas las respuestas incluyen explicación y justificación. Sintaxis OWL en Manchester donde se solicita.

Problema 1. Lógica de predicados

a) Representación lógica

Vocabulario utilizado:

- `Veterinario(x)`: x es veterinario.
- `Animal(x)`: x es animal.
- `AnimalRescatado(x)`: x es animal rescatado.
- `Atiende(x, y)`: x atiende a y.
- `Vacunado(x)`: x ha recibido una vacuna.

1. Todos los veterinarios atienden al menos a un animal.

`∀x (Veterinario(x) → ∃y (Animal(y) ∧ Atiende(x, y)))`

Justificación:

- ∀x porque es una afirmación universal sobre todos los veterinarios.
- Se usa implicación → y no conjunción, porque solo se exige algo a quienes cumplen `Veterinario(x)`.
- ∃y porque basta con que exista al menos un animal atendido, no todos.
- `Animal(y) ∧ Atiende(x, y)` garantiza que ese individuo sea animal y que exista la relación de atención.

2. Existe al menos un animal rescatado que ha recibido una vacuna.

`∃x (AnimalRescatado(x) ∧ Vacunado(x))`

Justificación:

- ∃x porque basta un caso para que la afirmación sea verdadera.
- Se usa conjunción ∧ porque el mismo individuo debe cumplir ambas propiedades a la vez.
- No se usa implicación, porque no es una regla general sino la afirmación de existencia de un individuo con dos características.

b) Identificación y corrección de errores

Afirmación a representar: *"Todos los animales que reciben una vacuna son animales rescatados"*.

Expresión propuesta:

`∀x (Vacunado(x) ∧ AnimalRescatado(x))`

Por qué es incorrecta:

1. Usa conjunción ∧ con cuantificador universal. Eso afirma que **todo individuo del universo** cumple ambas cosas a la vez: todo es vacunado y todo es animal rescatado.
2. No expresa condicionalidad. La frase original es un "si ... entonces ...": *si* está vacunado, *entonces* es rescatado. La fórmula propuesta no tiene → y por tanto no restringe, sino que generaliza en exceso.
3. Bastaría un contraejemplo (una persona no vacunada, un animal no rescatado) para hacer falsa la fórmula, aunque la afirmación original podría seguir siendo verdadera.

Expresión corregida:

`∀x (Vacunado(x) → AnimalRescatado(x))`

Variante más explícita delimitando a animales:

`∀x ((Animal(x) ∧ Vacunado(x)) → AnimalRescatado(x))`

Justificación de la corrección: con → solo se exige `AnimalRescatado(x)` a quienes cumplen el antecedente. Los individuos no vacunados no falsan la fórmula, que es exactamente lo que pide el enunciado.

Problema 2. Diseño de una ontología — Repositorio universitario

Dominio: organizar publicaciones para clasificar y consultar; distinguir digitales e impresas, registrar autores, identificar formatos, reconocer autoría múltiple y licencia abierta.

a) Clases necesarias y propósito

Clase	Propósito
`Publicacion`	Clase central. Toda obra del repositorio.
`PublicacionDigital`	Publicaciones en soporte digital. Permite clasificar y aplicar reglas propias de lo digital.
`PublicacionImpresa`	Publicaciones en soporte físico. Incompatible con la anterior.
`PublicacionMultiautor`	Clase definida para las publicaciones con ≥2 autores distintos. Se infiere automáticamente.
`PublicacionDigitalPDF`	Clase definida para publicaciones digitales disponibles en PDF. Se infiere automáticamente.

Clase	Propósito
`Autor`	Personas que escriben publicaciones. Se vincula vía `escritoPor`.
`Formato`	Formatos de disponibilidad. Ej.: `PDF`, `EPUB`, `HTML`, `TapaBlanda`, `TapaDura`.
`Licencia`	Licencias de uso de la publicación.
`LicenciaAbierta`	Subclase de `Licencia` para acceso abierto (ej. Creative Commons). Permite filtrar qué se puede difundir libremente.

Individuos ejemplo (a usar en Protégé, Problema 4): `pub1`, `pub2`, `autor1`, `autor2`, `formatoPDF`.

b) Relaciones entre conceptos

Propiedad (dominio → rango)	Qué información representa
`escritoPor` (`Publicacion → Autor`)	Autoría. Quién escribió cada publicación. Base para `PublicacionMultiautor`. Inversa: `esAutorDe`.
`disponibleEn` (`Publicacion → Formato`)	En qué formato(s) está disponible cada publicación. Base para `PublicacionDigitalPDF`.
`tieneLicencia` (`Publicacion → Licencia`)	Bajo qué licencia se publica. Permite identificar licencia abierta.

Se modelan como `ObjectProperty`, no como cadenas de texto, para poder razonar sobre los individuos relacionados y mantener integridad referencial.

c) Jerarquía de clases y restricciones

Representación textual:

```
Publicacion
  ├─ PublicacionDigital
  ├─ PublicacionImpresa
  ├─ PublicacionMultiautor      [definida]
  └─ PublicacionDigitalPDF      [definida]

Autor
Formato
  ├─ (individuos: PDF, EPUB, HTML, TapaBlanda, ...)
Licencia
  └─ LicenciaAbierta
```

Restricciones (Manchester):

```
1. PublicacionDigital SubClassOf Publicacion
2. PublicacionImpresa SubClassOf Publicacion
```

```
3. PublicacionDigital DisjointWith PublicacionImpresa
4. PublicacionMultiautor EquivalentTo Publicacion and (escritoPor min 2 Autor)
5. PublicacionDigitalPDF EquivalentTo PublicacionDigital and (disponibleEn value PDF)
   // alternativa con clase: ... and (disponibleEn some FormatoPDF)
6. Publicacion SubClassOf (escritoPor some Autor)
7. Publicacion SubClassOf (disponibleEn some Formato)
8. LicenciaAbierta SubClassOf Licencia
9. PublicacionConLicenciaAbierta EquivalentTo Publicacion and (tieneLicencia some Lic
```

Coherencia que aportan:

- 1-3 garantizan la distinción digital/impreso y la incompatibilidad exigida en el Problema 4.
- 4-5 son clases definidas (`EquivalentTo`), permiten clasificación automática por HermiT.
- 6-7 evitan publicaciones sin autor ni formato.
- 9 permite consultar qué se puede difundir en abierto.

d) Justificación de dos decisiones de diseño

Decisión 1: `Formato` como clase con individuos (`PDF`, …) y no como `xsd:string`. Si `formato` fuera un dato literal (`titulo`, `"PDF"`), no se podría usar `disponibleEn value PDF` ni `some FormatoPDF` para definir `PublicacionDigitalPDF`, ni distinguir formatos digitales de impresos con jerarquía. Con clase, el razonador puede inferir y se puede extender (nuevo formato = nuevo individuo, sin cambiar el esquema).

Decisión 2: `PublicacionMultiautor` y `PublicacionDigitalPDF` como clases definidas (`EquivalentTo`) y no como subclases primitivas (`SubClassOf`). Si fueran primitivas, habría que clasificar manualmente cada individuo y se podría olvidar o equivocar. Como definidas (condición necesaria y suficiente), HermiT clasifica solo cuando se cumplen `min 2` autores o `Digital + PDF`. Sin `EquivalentTo` no habría inferencia automática, que es justo lo que piden los requisitos funcionales del Problema 4.

Problema 3. Interpretación de axiomas OWL

a) Subclases

```
PublicacionDigital SubClassOf Publicacion
```

Significa que todo individuo de `PublicacionDigital` es también individuo de `Publicacion`. Es inclusión de conjuntos, no igualdad: puede haber publicaciones que no sean digitales (las impresas). La subclase hereda las restricciones de la superclase: si `Publicacion` exige `escritoPor some Autor`, toda digital también debe cumplirlo.

b) Restricciones `some` vs `only`

- `escritoPor some Autor` (existencial / `someValuesFrom`): Garantiza que **existe al menos un** valor de `escritoPor` que es `Autor`. No garantiza que *todos* lo sean, ni cuántos hay. Una publicación con un autor válido y otros valores no-autor seguiría cumpliéndola.

- `escritoPor only Autor` (universal / `allValuesFrom`): Garantiza que **todos** los valores de `escritoPor`, si existen, son de tipo `Autor`. No garantiza existencia: si la publicación no tiene ningún `escritoPor`, la restricción se cumple por vacío (verdad vacuosa).

En la práctica se combinan: `escritoPor some Autor` para exigir autor, `escritoPor only Autor` para prohibir valores no-autor.

c) Cardinalidad

1. Definición OWL adecuada (Manchester):

```
Class: PublicacionMultiautor
  EquivalentTo: Publicacion and (escritoPor min 2 Autor)
```

Se usa `EquivalentTo` (no solo `SubClassOf`) para que el razonador infiera automáticamente la pertenencia cuando se cumple el mínimo. `min 2` exige al menos dos valores distintos en `Autor`.

2. Por qué dos nombres no bastan:

OWL **no asume nombre único (UNA)**. Dos IRI distintos (`autor1`, `autor2`) pueden denotar al mismo individuo salvo que se declare lo contrario. El razonador puede fusionarlos y contar 1 solo autor.

Para que cuenten como 2 se debe declarar:

```
DifferentIndividuals: autor1, autor2
```

o bien:

```
autor1 DifferentFrom autor2
```

Sin esto, `escritoPor min 2` podría no dispararse aunque se hayan registrado dos nombres.

d) Equivalencia de clases

`EquivalentTo` define condición **necesaria y suficiente**. No solo dice "todo miembro cumple esto" (como `SubClassOf`), sino también "todo lo que cumple esto es miembro". Eso permite al razonador **clasificar automáticamente** individuos que no fueron tipados explícitamente.

Ejemplo del repositorio:

```
Class: PublicacionDigitalPDF
  EquivalentTo: PublicacionDigital and (disponibleEn value PDF)
```

Si se registra `pub1 type PublicacionDigital` + `pub1 disponibleEn PDF`, sin decir que es `PublicacionDigitalPDF`, HermiT lo infiere como miembro. Con solo `SubClassOf` esa inferencia no ocurriría.

Problema 4. Construcción y razonamiento en Protégé

Ontología construida desde cero en Protégé 5.6.9. IRI: `http://example.org/repositorio-universitario`. Archivo de entrega: `repositorio-universitario.owl` (mismo modelo del Problema 2).

a) Modelo construido e individuos registrados

Clases: `Publicacion`, `PublicacionDigital`, `PublicacionImpresa`, `PublicacionMultiautor` (definida), `PublicacionDigitalPDF` (definida), `Autor`, `Formato`, `Licencia`, `LicenciaAbierta`. Propiedades: `escritoPor`, `disponibleEn`, `tieneLicencia`, `titulo`. Axiomas clave (Manchester):

```
PublicacionDigital DisjointWith PublicacionImpresa
PublicacionMultiautor EquivalentTo Publicacion and (escritoPor min 2 Autor)
PublicacionDigitalPDF EquivalentTo PublicacionDigital and (disponibleEn value formato
DifferentIndividuals: autor1, autor2, autor3
```

Individuos de prueba:

Individuo	Tipo asertado	Hechos
`pub1_TesisIA`	`PublicacionDigital`	`escritoPor autor1, autor2`; `disponibleEn formatoPDF`; `tieneLicencia licCC_BY`; `titulo "Tesis de IA aplicada"`
`pub2_ArticuloRedes`	`PublicacionDigital`	`escritoPor autor1`; `disponibleEn formatoEPUB`
`pub3_LibroHistoria`	`PublicacionImpresa`	`escritoPor autor3`; `disponibleEn formatoTapaBlanda`

Vista general en Protégé (67 axiomas, 10 clases, 3 propiedades de objeto, 10 individuos, 2 `EquivalentClasses`, 1 `DisjointClasses`):

b) Inferencias esperadas antes de ejecutar HermiT

#	Inferencia esperada	Condición que la permite
1	`pub1_TesisIA` es `PublicacionMultiautor`	Tiene 2 autores (`autor1`, `autor2`) declarados distintos (`DifferentIndividuals`) → cumple `escritoPor min 2 Autor`.
2	`pub1_TesisIA` es `PublicacionDigitalPDF`	Es `PublicacionDigital` y `disponibleEn formatoPDF` → cumple la equivalencia con `hasValue formatoPDF`.

#	Inferencia esperada	Condición que la permite
—	pub2 NO debe clasificarse en ninguna de las dos	Solo 1 autor (no llega a `min` 2) y su formato es EPUB, no PDF.
—	pub3 NO debe clasificarse en ninguna de las dos	Es impresa (no digital) y tiene 1 solo autor.

Razonador seleccionado: HermiT 1.4.3.456 (Reasoner → HermiT → Start reasoner):

c) Resultados del razonador

Protégé muestra en amarillo los tipos inferidos de `pub1_TesisIA` (`PublicacionDigitalPDF`, `PublicacionMultiautor`), junto a sus aserciones (`disponibleEn formatoPDF`, `escritoPor autor1/autor2`):

Comprobación con el mismo motor HermiT por CLI (`-k` = ontología satisfacible; tabla de tipos asertados vs. inferidos; `True/True/False/False` según lo previsto en (b)):

Explicación: la inferencia 1 se produce por la clase definida con cardinalidad mínima (`min` 2) combinada con `DifferentIndividuals` (sin individuos distintos, el razonador podría fusionar `autor1` y `autor2` y no dispararía). La inferencia 2 se produce por la clase definida con `hasValue formatoPDF` sobre una publicación digital. `pub2` no dispara ninguna porque no satisface ninguna condición suficiente.

d) Prueba de incompatibilidad

Procedimiento: se tipó `pub3_LibroHistoria` (impresa) también como `PublicacionDigital`, quedando en dos clases declaradas disjuntas, y se ejecutó HermiT. Resultado: `InconsistentOntologyException: Inconsistent ontology` (ver Cap6, parte inferior). Explicación: un individuo no puede pertenecer a la vez a dos clases disjuntas; es lógicamente imposible, por eso HermiT declara la ontología inconsistente y se niega a razonar hasta corregir el modelo (en Protégé esto corresponde al aviso de ontología inconsistente y a la opción Reasoner → Explain inconsistent ontology).

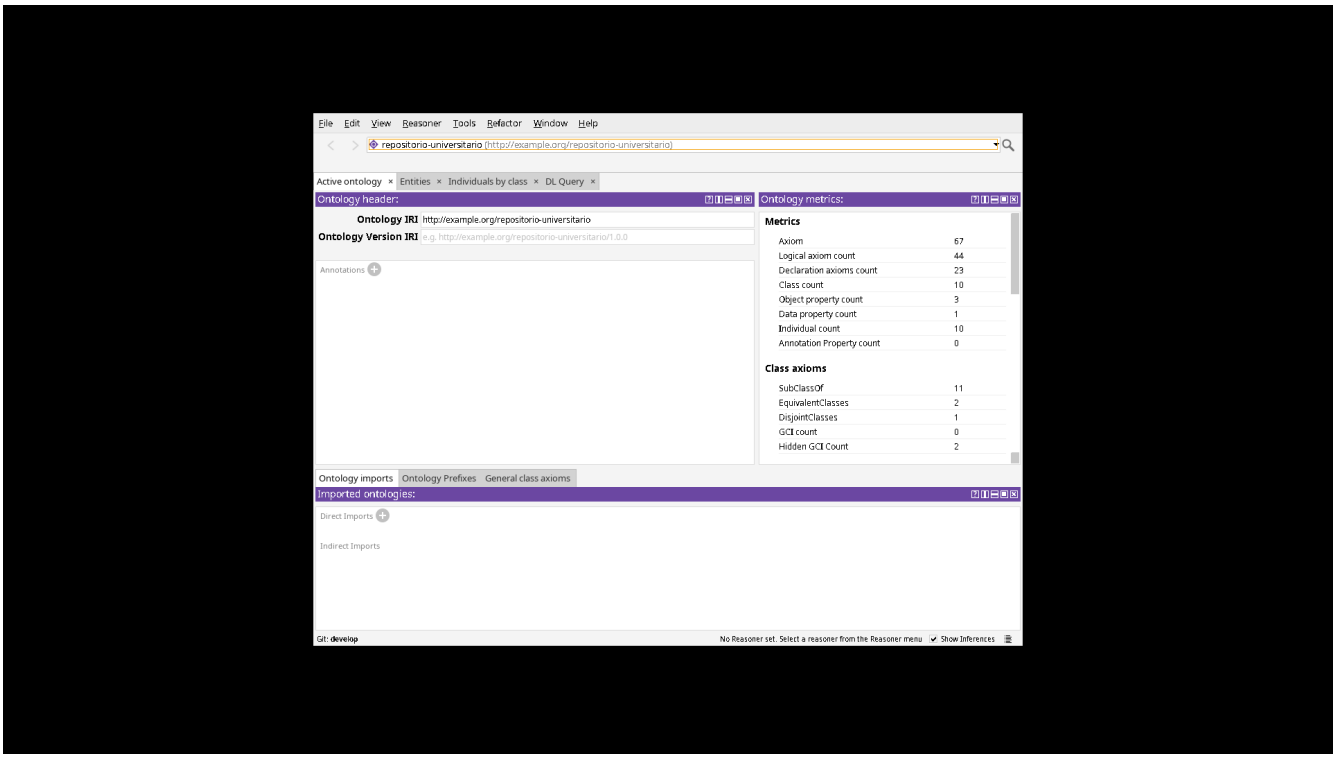


Figure 1: Cap1: ontología cargada en Protégé

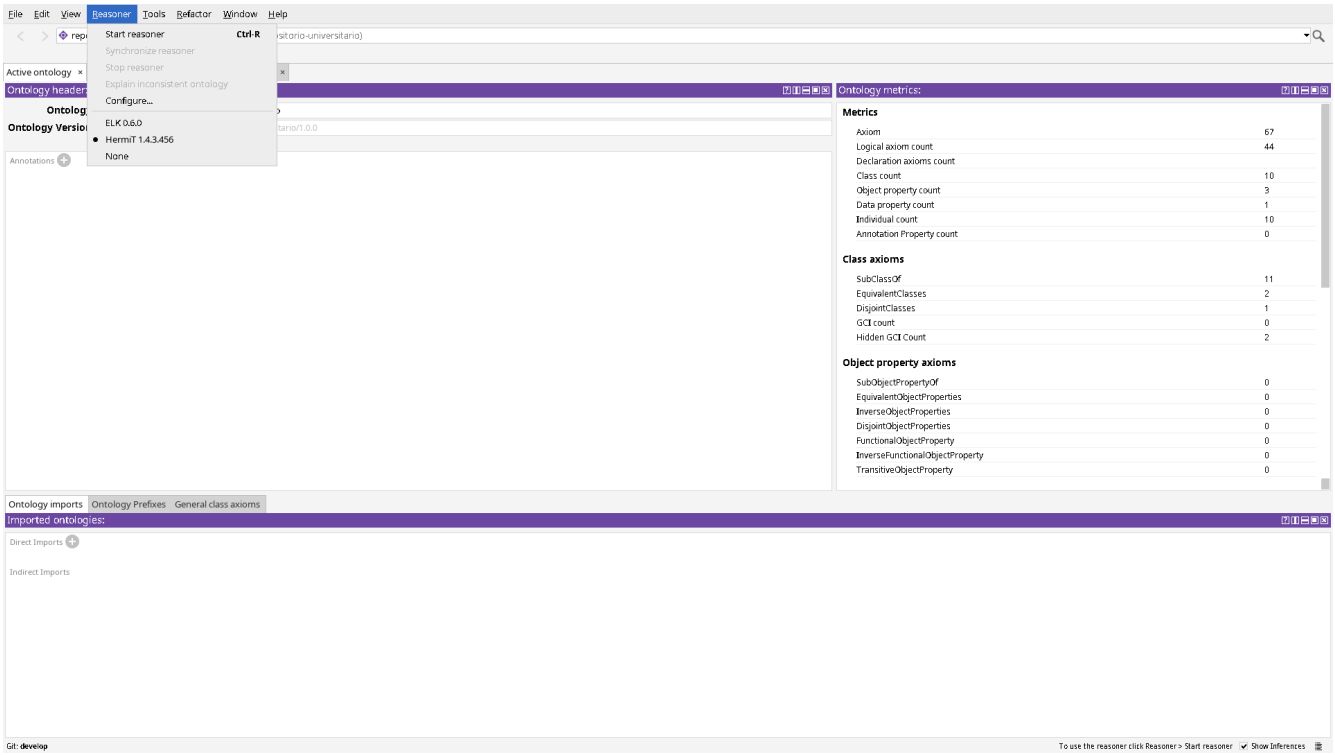


Figure 2: Cap2: HermiT seleccionado

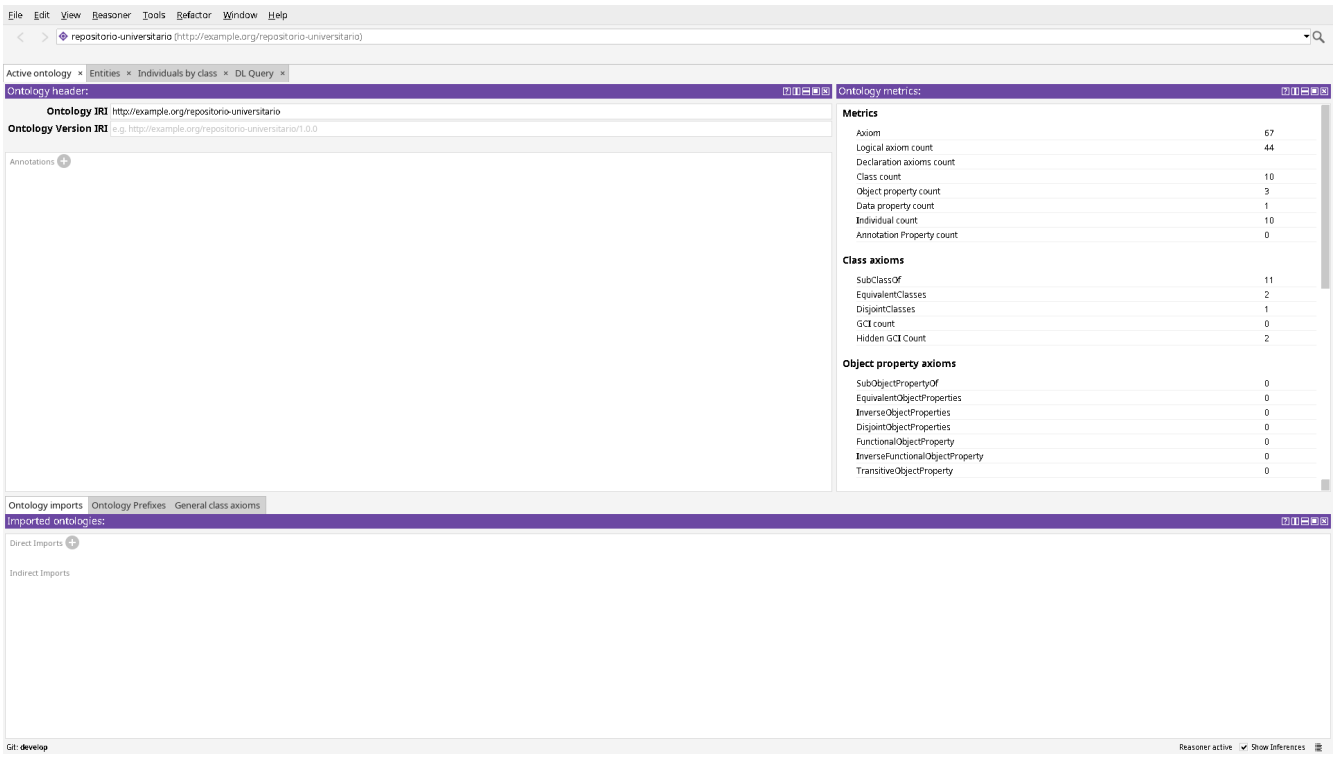


Figure 3: Cap3: razonador activo

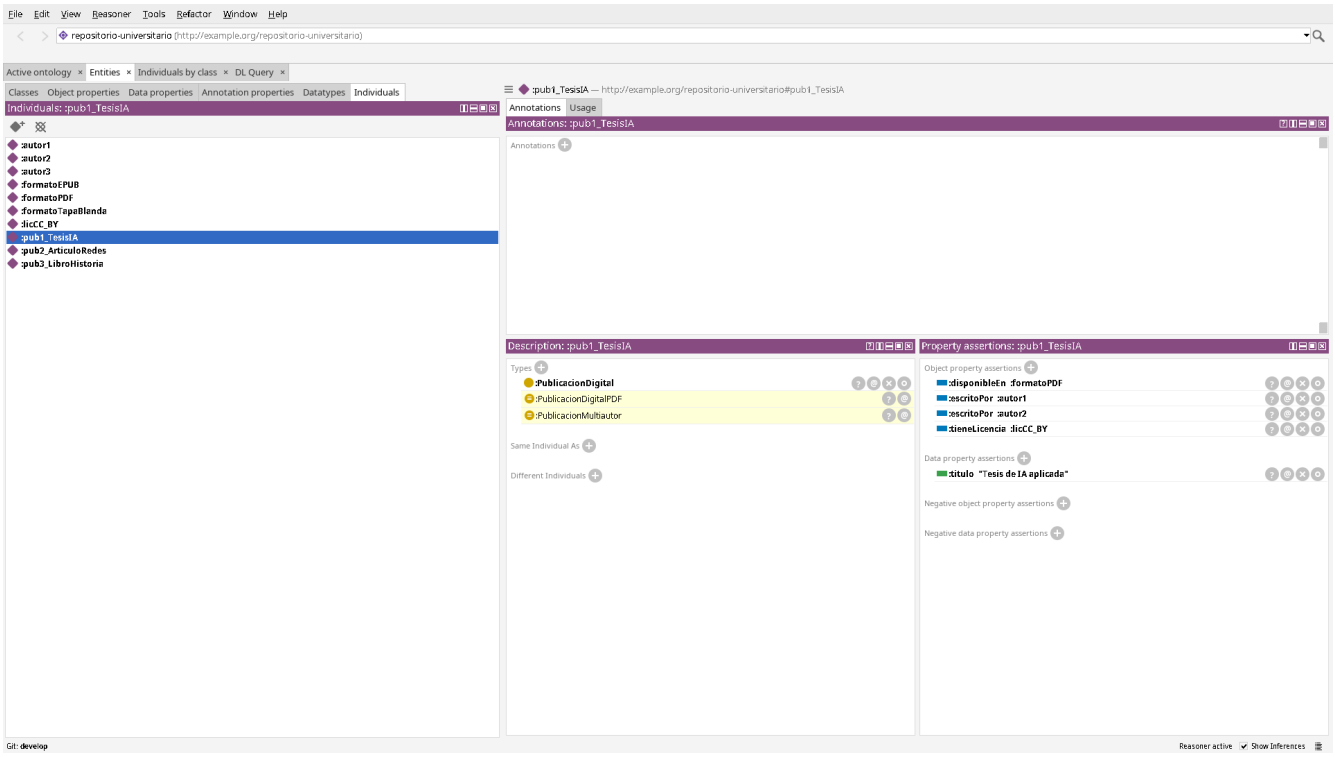


Figure 4: Cap4: inferencias de pub1 en Protégé con HermiT



Figure 5: Cap6: HermiT en terminal, consistencia e inferencias